

04. live ROS2 launch 入口

solar_ws0129-main

2026-07-29

Contents

1 対象	2
2 このファイルを読む理由	2
3 呼び出し関係	2
3.1 どこから呼ばれるか	2
3.2 次に読むと理解が進むファイル	2
3.3 同一リポジトリ内の主要依存	2
4 ソース冒頭の把握	2
4.1 代表コード断片	2
5 主要構造の読み取り	3
5.1 主要クラス・関数	3
5.2 ファイルを上から読んだときの定義順	3
5.3 import 群の詳細	3
6 このファイルを読む前に必要な基礎知識	4
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	4
6.2 名前、代入、型、参照	4
6.3 関数、引数、戻り値、スコープ	4
6.4 module、package、import、console_scripts	5
6.5 例外、try/finally、with、資源解放	5
6.6 launch Action、Node Action、実行可能名、remapping	5
6.7 class、独自クラス、インスタンス、self、継承	6
7 関数・クラスを上から順に解説	6
7.1 L10 関数 _setup	6
7.2 L16 関数 generate_launch_description	7
8 処理の流れ	8
9 このファイルの位置づけ	8

1 対象

項目	内容
ファイル	launch/solar_race_live.launch.py
ソース SHA-256	5b7604b18712958285819b64c5a5dcd7b3206c77f32136a4ac97e3a43263476f
種別	ROS 2 launch Python
区分	launch
役割	WiFi 文字列 bridge を使わない live 運用の基本 launch。
起動文脈	live モード起動時に ros2 launch される。

2 このファイルを読む理由

WiFi 文字列 bridge を使わない live 運用の基本 launch。

- profile を読み build_live_nodes に委譲する。
- ノード構成の本体は live_launch.py にある。

3 呼び出し関係

3.1 どこから呼ばれるか

- scripts/solar_control.sh
- SolarSim.ps1

3.2 次に読むと理解が進むファイル

- mpc_solarcar/live_launch.py
- mpc_solarcar/mpc_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/live_launch.py
- mpc_solarcar/solar_profile.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from mpc_solarcar.solar_profile import load_profile

def _setup(context):
    profile_yaml = LaunchConfiguration("profile_yaml").perform(context)
```

```

profile_path, cfg = load_profile(profile_yaml)
return build_live_nodes(profile_path, cfg, use_wifi=False)

def generate_launch_description():
    return LaunchDescription(
        [
            DeclareLaunchArgument("profile_yaml", default_value="config/solar/bwsc_2027_demo
.yaml"),
            OpaqueFunction(function=_setup),
        ]
    )

```

5 主要構造の読み取り

主要関数は `generate_launch_description`。

5.1 主要クラス・関数

行	種別	名前	補足
10	function	<code>_setup</code>	args: context
16	function	<code>generate_launch_description</code>	

5.2 ファイルを上から読んだときの定義順

行	内容
10	関数 <code>_setup</code> を定義する。
16	関数 <code>generate_launch_description</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>from launch import LaunchDescription</code>	<code>launch</code> が実行すべき action 群をまとめるため。このファイル内での主な使用位置は L17。
4	<code>from launch. actions import DeclareLaunchArgument, OpaqueFunction</code>	<code>DeclareLaunchArgument</code> や <code>OpaqueFunction</code> など <code>launch</code> action を使うため。このファイル内での主な使用位置は L19, L20。
5	<code>from launch. substitutions import LaunchConfiguration</code>	<code>launch</code> 引数の実行時値を参照するため。このファイル内での主な使用位置は L11。
6	<code>from mpc_solarcar.live_ launch import build_live_ nodes</code>	live 系 node 構成ビルダから <code>build_live_nodes</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/live_launch.py</code> 。このファイル内での主な使用位置は L13。

7	<code>frommpc_solarcar.solar_profileimportload_profile</code>	profile YAML 読込と検証から <code>load_profile</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主な使用位置は L12。
---	---	--

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘`float(...)`’、‘`int(...)`’、‘`bool(...)`’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘`None`’は値が無いことを示す単一のオブジェクト、‘`math.nan`’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘`is None`’と ‘`math.isfinite`’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘`def`’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘`f`’は関数そのもの、‘`f()`’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘`*args`’、‘`**kwargs`’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが `'self.z'` を更新すればオブジェクトの状態が残るが、単に `'z'` を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。`'from .model import SolarCarModel'` の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。`'def'` や `'class'` は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の `'console_scripts'` は、端末で使う実行可能名と `'package.module:function'` を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。`'try/except'` は想定した異常を処理し、`'finally'` は成功・失敗にかかわらず後始末を行う。

`'with open(...) as f:'` は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

`'except Exception: pass'` はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 launch Action、Node Action、実行可能名、remapping

`'launch_ros.actions.Node(...)'` は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

`'DeclareLaunchArgument'` は launch 実行時に受け取る入力欄を宣言し、`'LaunchConfiguration'` はその値を後で解決する substitution を表す。`'perform(context)'` は実行時 context から確定文字列を取り出す。

launch の `'name'` は Node 名 override、`'parameters'` は起動時 parameter、`'remappings'` は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

6.7 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

7 関数・クラスを上から順に解説

7.1 L10 関数 _setup

- 行範囲: L10–L13
- 所属: module 直下
- 定義: _setup(context)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: LaunchConfiguration, build_live_nodes, load_profile, perform
- 戻り値の要点: build_live_nodes(profile_path, cfg, use_wifi=False)
- 主なローカル代入名: cfg, profile_path, profile_yaml
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `profile_yaml` に `LaunchConfiguration('profile_yaml').perform(context)` の結果を代入する。
2. `(profile_path, cfg)` に `load_profile(profile_yaml)` の結果を代入する。
3. `build_live_nodes(profile_path, cfg, use_wifi=False)` を返す。

代表コード断片

```
def _setup(context):
    profile_yaml = LaunchConfiguration("profile_yaml").perform(context)
    profile_path, cfg = load_profile(profile_yaml)
    return build_live_nodes(profile_path, cfg, use_wifi=False)
```

7.2 L16 関数 `generate_launch_description`

- 行範囲: L16–L22
- 所属: module 直下
- 定義: `generate_launch_description()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DeclareLaunchArgument`, `LaunchDescription`, `OpaqueFunction`
- 戻り値の要点: `LaunchDescription([DeclareLaunchArgument('profile_yaml', default_value='config/solar/bwsc_2027_demo.yaml'), OpaqueFunction(function=_setup)])`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `LaunchDescription([DeclareLaunchArgument('profile_yaml', default_value='config/solar/bwsc_2027_demo.yaml'), OpaqueFunction(function=_setup)])` を返す。

代表コード断片

```
def generate_launch_description():
    return LaunchDescription(
        [
            DeclareLaunchArgument("profile_yaml", default_value="config/solar/bwsc_2027_demo
.yaml"),
            OpaqueFunction(function=_setup),
        ]
    )
```

8 処理の流れ

1. launch 引数を受け取り、profile を読み込む。

9 このファイルの位置づけ

この資料群では、`launch/solar_race_live.launch.py` を **launch** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。